دانشگاه تهران
دانشکدگان فنی

برنامه‌سازی پیشرفته

راهنمای درس

اشکال‌زدایی، جستجو و هوش مصنوعی

چگونه با خطاها روبه‌رو شویم

پیش از شروع

خطا بخشی طبیعی، اجتناب‌ناپذیر و حتی آموزنده از فرآیند برنامه‌نویسی است. هیچ برنامه‌نویسی، از مبتدی تا حرفه‌ای، کدی بدون خطا نمی‌نویسد؛ تفاوت در نحوهٔ برخورد با آن است. کامپایلر و محیط اجرای Java دشمن شما نیستند؛ پیام‌های خطا تلاش آن‌ها برای برقراری ارتباط و نشان‌دادن ریشهٔ مشکل است.

هدف این راهنما آموزش چهار مهارت زنجیره‌ای و متوالی است:

۱. **شناخت ماهیت خطاها:** تفکیک خطای کامپایلر، خطای زمان اجرا و خطای منطقی.
۲. **خواندن و کالبدشکافی خطاها:** استخراج اطلاعات حیاتی از متن پیام‌ها بدون سردرگمی.
۳. **جستجوی هدفمند:** فرمول‌بندی عبارات جستجو برای رسیدن سریع به پاسخ در منابع معتبر.
۴. **بهره‌گیری هوشمندانه از هوش مصنوعی:** پرامپت‌نویسی اصولی، درک محدودیت‌ها، بازبینی کد و فرآیند گام‌به‌گام رفع اشکال.

کدهای این راهنما از مفاهیم هفته‌های اول تا سوم درس (نوع‌های پایه، عملگرها، عبارت‌ها، حلقه‌ها، عبارات شرطی و بستر گرافیکی ساده) انتخاب شده‌اند و از هیچ مفهوم شیءگرایی استفاده نمی‌کنند.

کدهای نمونهٔ اشکال‌زدایی در گیت‌هاب ↗
GitHub · سه برنامه با خطاهای کامپایلر، زمان اجرا و منطقی

بخش ۱: خطاها چیستند؟ دسته‌بندی سه‌گانه

هنگام نوشتن و اجرای یک برنامه، اشکال‌ها می‌توانند در سه لایه و زمان کاملاً مجزا بروز کنند. پیش از تلاش برای اصلاح هر خطایی، باید بدانید با کدام دسته روبه‌رو هستید:

دستهٔ خطا	زمان وقوع	چه کسی متوجه می‌شود؟
خطای کامپایلر (Compile Error)	پیش از اجرای برنامه (زمان ترجمه)	کامپایلر (javac)
خطای زمان اجرا (Runtime Error)	هنگام اجرای برنامه	ماشین مجازی جاوا (JVM)
خطای منطقی (Logic Error)	برنامه تا پایان اجرا می‌شود	تنها برنامه‌نویس یا کاربر

- **خطای کامپایلر:** قواعد دستوری زبان (Syntax) یا سازگاری نوع‌ها (Types) نقض شده است. کامپایلر اجازهٔ تولید فایل اجرایی را نمی‌دهد. مثل این است که در جمله‌ای فارسی، فعل را فراموش کنید یا املای کلمه‌ای اشتباه باشد.
- **خطای زمان اجرا:** برنامه کامپایل شده و اجرا آغاز می‌شود، اما در میان راه دستور ناممکنی پیش می‌آید (مانند تقسیم عددی بر صفر یا درخواست حافظهٔ نامعتبر)؛ برنامه ناگهان متوقف و اصطلاحاً «کِرَش» (Crash) می‌کند.
- **خطای منطقی:** برنامه با موفقیت کامپایل و تا انتها اجرا می‌شود، اما خروجی آن با خروجی مورد انتظار مطابقت ندارد. کامپایلر یا سیستم هیچ پیام خطایی نمی‌دهند، چون دستورها از نظر زبان معتبر بوده‌اند اما مقصود شما را محقق نکرده‌اند.

پیش‌بینی
اگر حلقه‌ای قرار باشد 4 بار تکرار شود ولی به اشتباه 5 بار اجرا شود، این باگ جزو کدام‌یک از این سه دسته است؟ چرا کامپایلر مانع اجرای برنامه نمی‌شود؟

بخش ۲: خطای کامپایلر و نحوهٔ خواندن آن

کامپایلر زبان جاوا (javac) بازرس سخت‌گیری است. وقتی خطای کامپایل دریافت می‌کنید، پیام ظاهرشده شامل اطلاعات دقیق دربارهٔ محل و علت خطا است.

کالبدشکافی پیام کامپایلر

فرض کنید می‌خواهیم درصد شارژ باقی‌ماندهٔ باتری را با داشتن شارژ فعلی و ظرفیت کل به صورت درصدی صحیح حساب کنیم:

```
class BatteryCalc {
    public static void main(String[] args) {
        int remaining = 7;
        int capacity = 20;
        int percent = (double) remaining / capacity * 100;
        System.out.println(percent);
    }
}
```

چون عبارت سمت راست به دلیل تبدیل (double) دارای نوع اعشاری است، ریختن آن در متغیر صحیح int بدون تبدیل صریح مجاز نیست. کامپایلر چنین خطایی نمایش می‌دهد:

```
src/examples/debugging/BatteryCalc.java:7: error: incompatible types: possible lossy
conversion from double to int
    int percent = (double) remaining / capacity * 100;
                                                  ^
1 error
```

آناتومی پیام خطا

`BatteryCalc.java:7` ← نام فایل و شمارهٔ خط دقیق رخداد خطا
`error: incompatible types` ← نوع خطا: عدم تطابق نوع متغیر با مقدار انتسابی
`possible lossy conversion` ← توضیح تکمیلی: خطر از بین رفتن قسمت اعشاری در تبدیل
`^` ← نشانگر مکان دقیق خطا در عبارت

قانون طلایی اول: با دیدن پیام کامپایلر مضطرب نشوید! ابتدا فقط نام فایل و شمارهٔ خط را بخوانید، به همان سطر در ویرایشگر خود بروید و نماد نشانه (^) را بررسی کنید.

پنج خطای رایج کامپایلر در مبانی برنامه‌سازی

در جدول زیر، خطاهای متداولی که در چند هفتهٔ آغازین با آن‌ها روبه‌رو می‌شوید همراه نمونهٔ نادرست و اصلاح‌شده آورده شده‌اند:

۱. نماد پیدا نشد (cannot find symbol)

این خطا معمولاً به علت غلط املایی در نام متغیر یا استفاده از متغیری خارج از دامنهٔ تعریف (Scope) رخ می‌دهد.

کد نادرست:

```
int remaining = 7;
System.out.println(remaning);
```

کد اصلاح‌شده:

```
int remaining = 7;
System.out.println(remaining);
```

۲. ناسازگاری نوع‌ها (incompatible types)

سعی در انتساب یک نوع دادهٔ بزرگتر یا ناسازگار به متغیری با نوع محدودتر.

کد نادرست:

```
int score = 19.5;
boolean active = 1;
```

کد اصلاح‌شده:

```
double score = 19.5;
boolean active = true;
```

۳. پایان پیش‌ازموعد فایل (`reached end of file while parsing`)
معمولاً به معنای فراموش کردن آکولاد بسته (}) برای یک حلقه، شرط یا کلاس است.

کد نادرست:

```
for (int i = 0; i < 3; i++) {
    System.out.println(i);
// آکولاد بسته فراموش شد
```

کد اصلاح‌شده:

```
for (int i = 0; i < 3; i++) {
    System.out.println(i);
}
```

۴. انتظار نقطه‌ویرگول (`';' expected`)
پایان هر دستور در جاوا نیازمند نقطه‌ویرگول (;) است.

کد نادرست:

```
int step = 10
step += 5;
```

کد اصلاح‌شده:

```
int step = 10;
step += 5;
```

۵. شروع نامعتبر عبارت (`illegal start of expression`)
پرانتزهای باز و بستهٔ ناهماهنگ، یا قرار گرفتن کلمات کلیدی در جایی نامناسب.

کد نادرست:

```
int total = (5 + 3));
```

کد اصلاح‌شده:

```
int total = (5 + 3);
```

ردگیری

در قطعه‌کد زیر، دو خطای کامپایلر وجود دارد. بدون اجرای کامپایلر، آن‌ها را بیابید، نام خطای احتمالی هریک را بنویسید و کد را اصلاح کنید:

```
int count = 0
double average = totl / count;
```

بخش ۳: خطای زمان اجرا و ردگیری پشته (Stack Trace)

خطاهای زمان اجرا زمانی رخ می‌دهند که کد شما از نظر گرامری سالم است، اما در جریان اجرا با وضعیت غیرممکن روبه‌رو می‌شود.

کالبدشکافی ردگیری پشته (Stack Trace)

در برنامهٔ زیر، قرار است میانگین اعدادی را تا رسیدن به نشانهٔ پایان (-1) محاسبه کنیم:

```
import java.util.Scanner;

class AverageCalc {
    public static void main(String[] args) {
        int total = 0;
        int count = 0;
        Scanner input = new Scanner(System.in);
        int value = input.nextInt();
        while (value != -1) {
            int avg = total / count; // برابر 0 است count در اولین تکرار!
            total += value;
            count++;
            value = input.nextInt();
        }
        System.out.println("Total: " + total);
    }
}
```

اگر کاربر ورودی 5 3 -1 را وارد کند، برنامه پیش از افزودن عدد، اقدام به تقسیم total / count می‌کند و متوقف می‌شود:

```
Exception in thread “main” java.lang.ArithmeticException: / by zero
    at AverageCalc.main(AverageCalc.java:12)
```

آناتومی پیام خطا

ArithmeticException ← نوع استثنا — خطای محاسباتی ریاضی در زمان اجرا
/ by zero ← پیام تفصیلی استثنا — تلاش برای تقسیم یک عدد صحیح بر صفر
AverageCalc.main ← متد و کلاسی که خطا در آن رخ داده است
AverageCalc.java:12 ← نام فایل و شمارهٔ خط دقیق توقف برنامه

قانون طلایی دوم: در گزارش‌های طولانی خطا (Stack Trace)، همیشه از بالاترین خط شروع کنید و نخستین خطی را بیابید که به فایل و خطِ نوشته‌شده توسط **شما** اشاره می‌کند. کدهای درونی سیستم یا کتابخانه‌ها معمولاً در خطوط بعدی قرار دارند.

سه خطای زمان اجرای رایج در ادامهٔ مسیر

۱. ArithmeticException: / by zero — تقسیم بر صفر در محاسبات اعداد صحیح.
۲. ArrayIndexOutOfBoundsException — مراجعه به اندیسی فراتر از طول آرایه (در مباحث هفته‌های آینده).
۳. NullPointerException — تلاش برای استفاده از متغیری ارجاعی که به هیچ شیءای اشاره نمی‌کند.

پیش‌بینی

در پیام خطای زیر که از یک برنامهٔ گرافیکی صادر شده، مشکل اصلی در کدام خط از کد شماست؟

```
Exception in thread “main” java.lang.ArithmeticException: / by zero
    at LighthouseAnimation.drawLighthouse(LighthouseAnimation.java:24)
    at LighthouseAnimation.run(LighthouseAnimation.java:10)
    at acm.program.GraphicsProgram.runHook(GraphicsProgram.java:312)
```

بخش ۴: خطای منطقی — برنامه‌ای که مؤدبانه اشتباه می‌کند!

خطاهای منطقی بدقلق‌ترین نوع باگ‌ها هستند، زیرا نه کامپایلر شکایتی دارد و نه سیستم متوقف می‌شود. برنامه ظاهراً سالم اجرا می‌شود، اما نتیجه اشتباه است.

نمونه: تختهٔ شطرنجی با یک سطر اضافه (Off-by-One)

می‌خواهیم یک تختهٔ شطرنجی 4 در 4 با نویسه‌های متنی بسازیم:

```
class Checkerboard {
    public static void main(String[] args) {
        int rows = 4;
        int cols = 4;
        for (int row = 0; row <= rows; row++) { // به علامت => دقت کنید!
            for (int col = 0; col < cols; col++) {
                System.out.print((row + col) % 2 == 0 ? "X " : ". ");
            }
            System.out.println();
        }
    }
}
```

خروجی مورد انتظار (4 سطر):

```
X . X .
. X . X
X . X .
. X . X
```

خروجی واقعی برنامه (5 سطر):

```
X . X .
. X . X
X . X .
. X . X
X . X .
```

ریشهٔ اشکال: شرط حلقهٔ بیرونی `row <= rows` نوشته شده است. با شروع از صفر، مقادیر `row` برابر با 0، 1، 2، 3 و 4 خواهند بود؛ یعنی دقیقاً 5 بار تکرار!

سه الگوی پرتکرار خطای منطقی

۱. خطای یک‌به‌یک (Off-by-One Error)

اشتباه در تعیین حد مرزی حلقه با استفاده از `<=` به‌جای `<` یا شروع شمارنده از 1 به‌جای 0. **راهبرد کشف:** حلقه را در مقادیر اولیه و نهایی شمارنده به‌صورت ذهنی ردگیری کنید.

۲. تبدیل نوع دیرهنگام در تقسیم صحیح (Late Casting)

محاسبهٔ نادرست (نتیجه `0.0`):

```
double ratio = (double) (3 / 4);
```

محاسبهٔ درست (نتیجه `0.75`):

```
double ratio = (double) 3 / 4;
```

در نمونهٔ اول، تقسیم دو عدد صحیح ابتدا انجام شده و حاصل `0` به `0.0` تبدیل می‌شود.

۳. منطق شرطی یا عملگر اشتباه

استفاده از `||` به‌جای `&&`، یا اشتباه در معکوس‌سازی با `!`. همچنین کوتاه‌مداری (Short-circuit) ممکن است باعث اجرا نشدن بخش موردنظر شود.

راهبرد اشکال‌زدایی: ردگیری روی کاغذ (Desk Checking / Tracing)

پیش از تغییر دادن تصادفی کدها، یک برگه و قلم بردارید:

۱. جدولی بکشید که ستون‌های آن متغیرها و شروط حلقه باشند.
۲. مقادیر را خط به خط مانند کامپیوتر جلو ببرید و در هر تکرار به‌روزرسانی کنید.
۳. اولین جایی که مقدار متغیر با انتظار شما مغایرت پیدا کرد، همان سطر منبع اشکال است.

تلاش کنید

اگر بخواهیم در کد تختهٔ شطرنجی، ابعاد تخته را 3 در 5 قرار دهیم و مشکل سطر اضافه را هم حل کنیم، چه تغییراتی در مقادیر و شروط حلقه‌ها نیاز است؟ خروجی را پیش از اجرا روی کاغذ ترسیم کنید.

بخش ۵: چگونه خطاهایمان را جستجو کنیم؟

هنگامی که پیام خطا یا رفتار برنامه را متوجه نمی‌شوید، موتورهای جستجو بهترین ابزار هستند — به شرط آنکه بدانید چه چیزی را تایپ کنید.

فرمول طلایی جستجوی فنی

یک جستجوی مؤثر از فرمول زیر پیروی می‌کند:

فرمول جستجوی استاندارد

```
java [error name or concept] [exact behavior or operation]
```

به این سه مثال توجه کنید:

- `java ArithmeticException division by zero in while loop`
- `java "cannot find symbol" variable declared in loop`
- `java integer division returns zero float cast`

ترفندهای حرفه‌ای جستجو

۱. **استفاده از گیومه برای پیام دقیق:** قرار دادن متن خطا داخل گیومه (`"..."`) به موتور جستجو می‌گوید دقیقاً دنبال همان عبارت بگردد: `java "possible lossy conversion from double to int"`

۲. **محدود کردن به مراجع معتبر:** افزودن `site:stackoverflow.com` نتایج را روی پرسش و پاسخ‌های فنی متمرکز می‌کند.

۳. **حذف اطلاعات خصوصی و تصادفی:** هرگز نام متغیرهای پروژه، نام فایل کلاستان، یا شماره خط‌های محلی خود را در جستجو قرار ندهید (مثلاً `AverageCalc.java` در اینترنت وجود ندارد!).

چگونه نتایج را بخوانیم و ارزیابی کنیم؟

نمونه‌ای از یک نتیجهٔ جستجو:

نمونهٔ نتایج جستجو در گوگل

1. java - Why does integer division truncate in Java? - Stack Overflow
stackoverflow.com/questions/4141682/...
In Java, when you divide two integers, the result is always an integer. To get a floating-point...
2. ArithmeticException: / by zero in Java explained - Baeldung
baeldung.com/java-division-by-zero
Learn what causes java.lang.ArithmeticException and how to prevent it using guards...

در سایت Stack Overflow:

- به نماد تیک سبز (پاسخ پذیرفته‌شده توسط پرسش‌گر) دقت کنید.
- به امتیاز مثبت (Upvotes) پاسخ‌ها نگاه کنید. گاهی پاسخی که بیشترین رأی را دارد بهتر از پاسخ پذیرفته‌شده است.
- به سال ارسال پاسخ توجه کنید؛ در کدهای بسیار قدیمی ممکن است روش‌های منسوخ استفاده شده باشد.

۵ الگوی جستجوی آماده

- ☐ `java [error name] [operation]` — جستجوی پایه برای یافتن نمونه‌های مشابه
- ☐ `java "exact error message" site:stackoverflow.com` — جستجوی عین پیام خطا در استک اورفلو
- ☐ `java [operator/statement] not working [observed result]` — رفع اشکال رفتار نامنتظره
- ☐ `java how to [intended task] without [restriction]` — یادگیری پیاده‌سازی سازگار با درس
- ☐ `java difference between [term A] and [term B]` — درک تفاوت دو مفهوم مشابه (مانند `for` و `while`)

پیش‌بینی

فرض کنید با خطای `';' expected` در تعریف یک متغیر مواجه شده‌اید اما مطمئنید در انتهای خط سمیکالن گذاشته‌اید. چه عبارتی برای جستجو پیشنهاد می‌کنید؟

بخش ۶: هوش مصنوعی چگونه و کجا می‌تواند کمک کند؟

مدل‌های زبانی بزرگ (ChatGPT, Claude, Gemini مانند) می‌توانند مانند یک دستیار آموزشی (TA) صبور در کنار شما باشند؛ اما مانند هر ابزار قدرتمندی، اگر روش استفاده از آن را ندانید، مهارت یادگیری شما را تضعیف خواهند کرد.

هوش مصنوعی چه کارهایی می‌تواند و نمی‌تواند بکند؟

کارهایی که در آن‌ها توانا است:

- توضیح دادن معنای یک پیام خطای مبهم به زبان ساده.
- بررسی یک قطعه‌کد کوتاه و اشاره به علت عدم کارکرد آن.
- پیشنهاد دادن مثال‌های ساده برای شفاف‌سازی یک مفهوم (مانند تفاوت `for` و `while`).
- پیدا کردن اشتباهات املایی و نشانه‌گذاری‌های فراموش‌شده.

محدودیت‌ها و خطرات:

- هوش مصنوعی ممکن است با اطمینان کامل کدی نادرست یا منسوخ به شما تحویل دهد (Hallucination).
- هوش مصنوعی از پیش‌نیازها و محدودیت‌های درس شما (مانند ممنوعیت استفاده از توابع پیشرفته یا کلاس‌ها) آگاه نیست مگر اینکه صریحاً قید کنید.
- کپی کردن مستقیم کد هوش مصنوعی بدون درک آن، در آزمون‌ها و تمرین‌های عملی بعدی باعث ناتوانی شما خواهد شد.

هشدار

هوش مصنوعی جایگزین مغز شما نیست؛ یک آینه است! کیفیت پاسخی که دریافت می‌کنید مستقیماً وابسته به کیفیت سوال و اطلاعاتی است که به آن می‌دهید.

قالب یک پرامپت (Prompt) حرفه‌ای

یک پرامپت استاندارد برای رفع اشکال باید سه رکن اصلی داشته باشد:

قالب پرامپت رفع اشکال

I am learning introductory Java (no OOP, no advanced libraries).
Here is my code (10 lines):
[کد کوتاه خود را اینجا قرار دهید]

What I expected: [خروجی یا رفتار مورد انتظار شما]
What actually happened: [پیام خطا یا خروجی فعلی]
Please explain what is causing the issue and guide me to fix it myself without writing the full solution.

مقایسهٔ عملی: پرامپت ضعیف در برابر پرامپت هدفمند

فرض کنید می‌خواهیم باگ اضافه بودن سطر در برنامهٔ تختهٔ شطرنجی را با کمک هوش مصنوعی حل کنیم:

پرامپت ضعیف و پاسخ سردرگم‌کننده

شما:

my java code doesn't work please fix it
[چسباندن ۱۰۰ خط کد مختلف از چند بخش پروژه]

AI:

There could be multiple reasons why your code isn't working. You might have issues with loop variables, scanner inputs, or class structure. Here is a completely rewritten version using Object-Oriented patterns, Streams, and 2D arrays...

اشکال: هوش مصنوعی کدی پیچیده با استفاده از کلاس‌ها و مفاهیمی تولید کرد که اصلاً هنوز نخوانده‌اید و مجاز به استفاده از آن‌ها نیستید!

پرامپت هدفمند و پاسخ آموزنده

شما:

I am learning basic Java loops (no OOP). I am trying to print a 4x4 checkerboard, but my program prints 5 rows instead of 4.

```
int rows = 4;
int cols = 4;
for (int row = 0; row <= rows; row++) {
    for (int col = 0; col < cols; col++) {
        System.out.print((row + col) % 2 == 0 ? "X " : ". ");
    }
    System.out.println();
}
```

Expected: 4 rows. Actual: 5 rows. Where is the extra iteration coming from?

AI:

The extra row is coming from the outer loop condition: `row <= rows`.
Since `row` starts at `0` and stops when it exceeds `rows` (4), the loop executes for `row = 0, 1, 2, 3, 4` — which is 5 iterations in total.
To get exactly 4 rows, change the condition from `<=` to `<`: `for (int row = 0; row < rows; row++)`
This is known as an "off-by-one error".

جدول بایدها و نبایدهای کار با هوش مصنوعی

معیار	روش درست ✓	روش اشتباه ✗
دامنهٔ کد	تنها بخش مشکل‌دار (5 تا 15 خط)	کپی کردن کل فایل و پروژه
توصیف وضعیت	بیان دقیق انتظار در مقابل واقعیت	«کدم کار نمی‌کنه» یا «درستش کن»
خطا	کپی کردن متن دقیق خطا یا استک تریس	«یک ارور قرمز رنگ می‌ده»
سطح پاسخ	درخواست راهنمایی و چرایی اشکال	درخواست نوشتن کل پروژه از ابتدا
صحت‌سنجی	تست و ارزیابی شخصی هر پیشنهاد	اعتماد کورکورانه و کپی مستقیم

چرخهٔ استاندارد حل مسئله (Debugging Workflow)

هنگامی که به مانعی برخوردید، دقیقاً این مسیر گام‌به‌گام را طی کنید:

۱. **گام اول (خواندن و تفکر):** پیام خطای کامپایلر یا زمان اجرا را با دقت بخوانید. شماره سطر را بیابید و متغیرها را بازبینی کنید.
۲. **گام دوم (آزمون فرضیه):** با تغییر کوچک و هدفمند، دوباره برنامه را اجرا کنید و بررسی کنید آیا پیام خطا تغییر کرد؟
۳. **گام سوم (جستجوی مستقل):** اگر علت خطا نامشخص بود، با فرمول بخش ۵ در وب جستجو کنید.
۴. **گام چهارم (همفکری با هوش مصنوعی):** اگر پاسخ مناسبی نیافتید، با پرامپت ساختاریافته از هوش مصنوعی بخواهید علت را برایتان باز کند.
۵. **گام پنجم (مشورت حضوری):** اگر پس از طی مراحل بالا مشکل پابرجا بود، با همراه داشتن گزارش دقیق کارهای انجام‌شده، به دستیار آموزشی یا استاد مراجعه کنید.

تلاش کنید

تصور کنید در حین نوشتن یک حلقهٔ شمارش با خطای زیر روبه‌رو شده‌اید:

```
Exception in thread “main” java.lang.ArithmeticException: / by zero
    at HarborStatus.main(HarborStatus.java:9)
```

یک پرامپت انگلیسی مطابق قالب این بخش بنویسید که بدون درخواست کد آماده، علت خطا را از هوش مصنوعی جویا شود.

مرور پایانی و خودآزمایی

پیش از شروع تکالیف بعدی، مطمئن شوید می‌توانید به پرسش‌های زیر به زبان خودتان و با استدلال پاسخ دهید:

۱. تفاوت اساسی میان خطای کامپایلر، خطای زمان اجرا و خطای منطقی چیست؟ چرا کامپایلر قادر به تشخیص خطای تقسیم بر صفر قبل از اجرا نیست؟
۲. در یک استک تریس (Stack Trace) چند ده سطری، اولین نگاه شما باید به کدام خط جلب شود؟
۳. در جستجوی اینترنتی برای یک باگ، چه تفاوت تعیین‌کننده‌ای میان کلمات داخل گیومه و کلمات بدون گیومه وجود دارد؟
۴. چرا فرستادن کل پروژه به هوش مصنوعی با پرامپت «کدم کار نمی‌کنه» معمولاً به کدهای پیچیده و غیرقابل استفاده ختم می‌شود؟

چک‌لیست مهارت‌های فردی

پیش از تحویل تمرین‌ها یا پرسیدن از دیگران، این معیارها را بررسی کنید:

- ☐ می‌توانم با دیدن پیام خطای کامپایلر، شمارهٔ سطر و نشانگر (^) را بیابم و علت اعلام‌شده را بفهمم.
- ☐ می‌توانم در استک تریس‌های طولانی، نخستین سطر مربوط به فایل خودم را جدا کنم و ریشهٔ توقف برنامه را شناسایی کنم.
- ☐ روش ردگیری روی کاغذ (Tracing) را برای حلقه‌ها و شرایط مرزی پیش از تغییر تصادفی کدها به کار می‌گیرم.
- ☐ می‌توانم برای هر ارور یا رفتار ناخواسته، یک عبارت جستجوی ساختاریافته به انگلیسی بسازم.
- ☐ هنگام استفاده از هوش مصنوعی، پرامپت سه‌بخشی (زمینه، کد کوتاه، مشکل دقیق) می‌نویسم و پاسخ را شخصاً راستی‌آزمایی می‌کنم.